EVT HARDWARE PROTOTYPING

23SDEE06A

STUDENT ID:

STUDENT NAME:

ACADEMIC YEAR: 2025-26

Table of Contents

A.Y. 2025-26 LAB AND SKILL CONTINUOUS EVALUATION

S.No	Date	Exercise Description	Pre-class exercises (10M)	In-class exercises (35M)			Viva Voce (5M)	Total (50M)	Faculty Signature
				Program (10M)	Experimentation (15M)	Analysis & Inference (10M)			
1.		Peripherals study & LED control							
2.		IR and Relay Interface							
3.		Ultrasonic sensor interface							
4.		Motor Control							
5.		Open CV functions							
6.		Object classification							
7.		Traffic sign recognition							
8.		Lane Detection							
9.		Route Visualization and logging							
10.		MPU 6050 logging							
11		GPS Coordinate logging							
12.		Pot hole Identification							
13.		Voice based Cruise Control							

Exercise No.		Date	

Familiarization of Raspberry-Pi5 Peripherals and LED control

Objective:

To familiarize with the Raspberry Pi 5 hardware, operating system, GPIO peripherals, and Python-based LED control for embedded prototyping applications

Hardware and Software requirements:

Raspberry Pi 5

Micro SD card (32 GB+)

LED, 330Ω resistor

Breadboard, jumper wires

Platform: Raspberry Pi OS (64-bit)

Python Libraries: gpiozero, RPi.GPIO, time, os

Pre-class exercises:

Topics

- Raspberry Pi 5 architecture
- GPIO pin numbering (BCM vs BOARD)
- Linux terminal basics

Questions

- What is the role of GPIO in embedded systems?
- Difference between microcontroller and microprocessor?
- Why is a current-limiting resistor required for LED?
- What is the default Python version in Raspberry Pi OS?

Exercise No.		Date	

In-Lab Exercise:

a) Peripheral Identification

- USB 3.0, CSI, DSI, Ethernet, GPIO header
- Power management IC and cooling

b) Algorithm

1. Configure GPIO pin as output
2. Set pin HIGH → LED ON
3. Delay
4. Set pin LOW → LED OFF

c) Important Code Snippet

```
from gpiozero import LED
from time import sleep
led = LED(17)
while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Implementation of IR sensor control of Relay using Raspberry-Pi5 GPIO Module

Objective:

To interface an IR proximity sensor with Raspberry Pi 5 and control a relay module based on obstacle detection.

Hardware and Software requirements:

Raspberry Pi 5

IR Sesnor, Rely Module, Breadboard, jumper wires

Platform: Raspberry Pi OS (64-bit)

Python Libraries: gpiozero, RPi.GPIO, time, os

Pre-class exercises:

Topics

- Digital sensors
- Relay operation
- Isolation and safety

Questions

- Why relays are used instead of GPIO directly?
- What is active-low relay?

Exercise No.		Date	

In-Lab Exercise:

Algorithm

1. Read IR sensor output
2. If obstacle detected → activate relay
3. Else → deactivate relay

Code Snippet

```
from gpiozero import DigitalInputDevice, OutputDevice
ir = DigitalInputDevice(23)
relay = OutputDevice(24, active_high=False)
while True:
    relay.value = ir.value
```

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Implementation of Ultrasonic sensor-based distance measurement and GUI display using Raspberry-Pi5 GPIO Module

Objective:

To measure distance using an ultrasonic sensor and display real-time values on a GUI.

Hardware and Software requirements:

Raspberry Pi 5

HC-SR04 ultrasonic sensor

Platform: Raspberry Pi OS (64-bit)

Python Libraries: tkinter, RPi.GPIO, time

Pre-class exercises:

Topics

- Time-of-flight principle
- GUI fundamentals

Questions

- Why echo pin requires input capture accuracy?
- Speed of sound significance?

Exercise No.		Date	

In-Lab Exercise:

Algorithm

1. Trigger ultrasonic pulse
2. Measure echo time
3. Calculate distance
4. Update GUI label

Key Code

distance = (pulse_time * 34300) / 2

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Implementation of Ultrasonic sensor- based motor direction and speed control using Raspberry-Pi5 GPIO Module

Objective:

To design and implement a distance-based DC motor direction and speed control system using an ultrasonic sensor and Raspberry Pi 5 GPIO for autonomous obstacle avoidance.

Hardware and Software requirements:

Raspberry Pi 5

L298N motor driver, Dc Motor, External Motor Power Supply (7–12 V)

Platform: Raspberry Pi OS (64-bit)

Python Libraries: RPi.GPIO, time

Pre-class exercises:

Topics

- Working principle of ultrasonic sensors
- H-bridge motor driver (L298N)
- PWM generation using GPIO
- Speed vs duty cycle relationship

Questions

- Why cannot DC motors be driven directly from Raspberry Pi GPIO?
- What is the function of ENA and ENB pins in L298N?
- How does PWM control motor speed?
- Why obstacle distance is critical in autonomous navigation?

Exercise No.		Date	

In-Lab Exercise:

a) Connection / Schematic Description

- Ultrasonic TRIG → GPIO output
- Ultrasonic ECHO → GPIO input
- L298N IN1, IN2 → Direction control pins
- ENA → PWM-enabled GPIO
- Motor power isolated from Pi supply

b) Detailed Algorithm

1. Initialize GPIO pins for ultrasonic sensor
2. Initialize motor driver pins and PWM
3. Send ultrasonic trigger pulse
4. Measure echo return time
5. Calculate distance using speed of sound
6. If distance > safe threshold
    - Move motor forward at high speed
7. If distance between warning threshold
    - Reduce motor speed
8. If distance < critical threshold
    - Stop or reverse motor
9. Repeat continuously

c) Important Code Snippets

```
from gpiozero import Motor, DistanceSensor
Sensor = DistanceSensor(echo=24, trigger=23)
motor = Motor(forward=17, backward=18, pwm=True)
while True:
   dist = sensor.distance * 100  # cm
   if dist > 50:
      motor.forward(0.8)
   elif dist > 20:
      motor.forward(0.4)
```

```
    else:
        motor.stop()
```

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Verification of Image and Video processing functions in Raspberry-Pi5 using Open CV library

Objective:

To capture images and videos using Raspberry Pi camera and verify basic image processing operations using OpenCV.

Hardware and Software requirements:

Raspberry Pi 5

USB camera

Platform: Raspberry Pi OS (64-bit)

Python Libraries: open cv-python, numpy

Pre-class exercises:

Topics

- Digital image representation
- Color spaces (RGB, Grayscale)
- Video frames and FPS

Questions

- Difference between image and video processing?
- Why grayscale conversion is preferred in CV?
- What is edge detection used for?

Exercise No.		Date	

In-Lab Exercise:

Algorithm

1. Initialize camera using OpenCV
2. Capture image frame
3. Convert to grayscale
4. Apply blur and edge detection
5. Display processed output

Code Snippet

```
import cv2
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
cv2.imshow("Edges", edges)
```

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Object Recognition and Classification using YOLO ML Model and Open CV through Raspberry-Pi5

Objective:

To detect and classify real-world objects using YOLO deep learning model on Raspberry Pi 5.

Hardware and Software requirements:

Raspberry Pi 5

USB camera

Platform: Raspberry Pi OS (64-bit)

Python Libraries: open cv-python, ultralytics, torch

Pre-class exercises:

Topics

- CNN basics
- Object detection vs classification
- YOLO architecture

Questions

- Why YOLO is preferred for real-time detection?
- What is confidence threshold?

Exercise No.		Date	

In-Lab Exercise:

Algorithm

- Load YOLO pre-trained model
- Capture video frame
- Run inference
- Extract bounding boxes and labels
- Display detected object

Code Snippet

```
from ultralytics import YOLO

import cv2

model = YOLO("yolov8n.pt")

results = model(frame)
```

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Traffic sign recognition using YOLO ML model and Open CV through Raspberry-Pi5

Objective:

To identify and classify traffic signs for autonomous driving decisions.

Hardware and Software requirements:

Raspberry Pi 5

USB camera

Platform: Raspberry Pi OS (64-bit)

Python Libraries: open cv-python, ultralytics, torch

Pre-class exercises:

Topics

- Traffic sign datasets
- Region of Interest (ROI)
- Speed sign semantics

Questions

- Why ROI improves accuracy?
- Difference between object and sign detection?

Exercise No.		Date	

In-Lab Exercise:

Algorithm

- Capture road image
- Apply ROI mask
- YOLO inference
- Map sign → vehicle action

Code Snippet

```
if label == "speed_limit_40":
    max_speed = 40
```

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Lane detection using classical approach and Open CV through Raspberry-Pi5

Objective:

To detect road lane boundaries using classical computer vision techniques.

Hardware and Software requirements:

Raspberry Pi 5

USB camera

Platform: Raspberry Pi OS (64-bit)

Python Libraries: open cv-python, ultralytics, torch

Pre-class exercises:

Topics

- Edge detection
- Hough Transform
- Perspective transformation

Questions

- Why Canny before Hough?
- What is ROI masking?

Exercise No.		Date	

In-Lab Exercise:

Algorithm

- Convert frame to grayscale
- Apply Gaussian blur
- Detect edges
- Apply ROI mask
- Detect lines using Hough Transform

Code:

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Pot hole identification using Open CV through Raspberry-Pi5

Objective:

To detect potholes on road surfaces using camera-based image processing techniques for road condition monitoring in autonomous vehicles.

Hardware and Software requirements:

Raspberry Pi 5

USB Camera

Platform: Raspberry Pi OS (64-bit)

Python Libraries: opencv-python, numpy,imutils

Pre-class exercises:

Topics

- Image thresholding
- Contour detection
- Texture and shadow analysis

Questions

- Why potholes appear darker in images?
- How lighting affects pothole detection?
- Why contour area filtering is required?

Exercise No.		Date	

In-Lab Exercise:

a) System Description

Camera captures road images → image processing → pothole region detection → bounding box marking.

b) Detailed Algorithm

- Capture frame from camera
- Convert image to grayscale
- Apply Gaussian blur
- Apply adaptive thresholding
- Detect contours
- Filter contours based on area and shape
- Mark pothole regions on image

c) Important Code Snippets

Frame Processing

```
import cv2

import numpy as np

cap = cv2.VideoCapture(0)

Pothole Detection Logic

ret, frame = cap.read()

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5,5), 0)

thresh = cv2.adaptiveThreshold(

   blur, 255,

   cv2.ADAPTIVE_THRESH_GAUSSIAN_C,

   cv2.THRESH_BINARY_INV, 11, 2)

contours, _ = cv2.findContours(

   thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for cnt in contours:

   area = cv2.contourArea(cnt)

   if area > 1000:

      x,y,w,h = cv2.boundingRect(cnt)

      cv2.rectangle(frame,(x,y),(x+w,y+h),(0,0,255),2)
```

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Real-Time Route Visualization and Coordinate Logging using OpenStreetMap on Raspberry Pi 5

Objective:

To integrate OpenStreetMap with Raspberry Pi 5 using Python, such that a given start and end location are visualized as a route on a web map and the intermediate path coordinates are logged into a file for autonomous navigation analysis.

Hardware and Software requirements:

Raspberry Pi 5

Wifi/Ethernet module

Platform: Raspberry Pi OS (64-bit)

Python Libraries: folium, geopy, osmnx, network, json, csv

Pre-class exercises:

Topics

- OpenStreetMap
- Latitude and longitude representation
- Geocoding and reverse geocoding
- Web-based map visualization (HTML maps)

Questions

- What is the difference between Google Maps and OpenStreetMap?
- Why are latitude and longitude required for navigation?
- What is a waypoint or intermediate coordinate?
- Why is browser-based visualization preferred over GUI in Raspberry Pi?
- How can route coordinates be used in autonomous vehicles?

Exercise No.		Date	

In-Lab Exercise:

a) System Description / Flow

- User provides start location and destination (address or coordinates)
- Python script:
    - Converts addresses to latitude/longitude
    - Fetches road network from OpenStreetMap
    - Generates a route between the two points
    - Displays the route on an interactive web map
    - Extracts intermediate coordinates
    - Logs them into a file (CSV/JSON)

b) Detailed Algorithm

1. Import required Python libraries
2. Accept start and end locations as text input
3. Convert locations into latitude and longitude using geocoding
4. Download the road network around the route region from OpenStreetMap
5. Generate a route between start and end points
6. Extract all intermediate latitude–longitude coordinates along the route
7. Create an interactive map using Folium
8. Plot the route and markers on the map
9. Save the map as an HTML file
10. Automatically open the map in the web browser
11. Store intermediate coordinates into a file for later use

c) Important Code Snippets

Geocoding Start & End Locations

```
from geopy.geocoders import Nominatim
geolocator = Nominatim(user_agent="rpi_av_lab")
start = geolocator.geocode("Bangalore Electronic City")
end = geolocator.geocode("Bangalore Silk Board")
start_coords = (start.latitude, start.longitude)
end_coords = (end.latitude, end.longitude)
```

Creating Route Map using OpenStreetMap

```
import osmnx as ox
import folium
G = ox.graph_from_point(start_coords, dist=3000, network_type='drive')
orig = ox.nearest_nodes(G, start.longitude, start.latitude)
dest = ox.nearest_nodes(G, end.longitude, end.latitude)
route = ox.shortest_path(G, orig, dest)
```

Extracting Intermediate Coordinates

```
route_coords = [(G.nodes[n]['y'], G.nodes[n]['x']) for n in route]
```

Visualizing Route on Web Map

```
m = folium.Map(location=start_coords, zoom_start=13)
folium.Marker(start_coords, popup="Start").add_to(m)
folium.Marker(end_coords, popup="End").add_to(m)
folium.PolyLine(route_coords, color="blue").add_to(m)
m.save("route_map.html")
```

Logging Coordinates to File

```
import csv
with open("route_coordinates.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Latitude", "Longitude"])
    for lat, lon in route_coords:
        writer.writerow([lat, lon])
```

Exercise No.		Date	

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Real-Time Accelerometer Data Acquisition and Logging using Raspberry Pi 5

Objective:

To interface an accelerometer sensor with Raspberry Pi 5 and log real-time acceleration data along X, Y, and Z axes for vehicle motion analysis and road condition monitoring.

Hardware and Software requirements:

Raspberry Pi 5

Accelerometer Sensor (MPU6050)

Platform: Raspberry Pi OS (64-bit) I2C enabled

Python Libraries: time, csv, math, smbus2

Pre-class exercises:

Topics

- Accelerometer working principle
- I^2C communication protocol
- Sensor axes and orientation
- Units of acceleration (g, m/s^2)

Questions

- What does an accelerometer measure?
- Difference between static and dynamic acceleration?
- Why I^2C is suitable for sensor interfacing?
- How does vehicle vibration reflect road condition?
- Why Z-axis acceleration is important in pothole detection?

Exercise No.		Date	

In-Lab Exercise:

a) Connection Description

- VCC → 3.3V
- GND → GND
- SDA → GPIO2 (SDA)
- SCL → GPIO3 (SCL)

b) Detailed Algorithm

1. Enable I²C interface on Raspberry Pi
2. Initialize I²C bus communication
3. Wake up accelerometer sensor from sleep mode
4. Read raw acceleration data from X, Y, Z registers
5. Convert raw values into acceleration in g
6. Display real-time acceleration values
7. Compute acceleration magnitude (optional)
8. Log acceleration data with timestamp into CSV file
9. Repeat continuously

c) Important Code Snippets

Initialize I²C and MPU6050

```
from smbus2 import SMBus

import time

import csv

import math

bus = SMBus(1)

MPU_ADDR = 0x68

PWR_MGMT_1 = 0x6B

bus.write_byte_data(MPU_ADDR, PWR_MGMT_1, 0)
```

Read Raw Accelerometer Data

```
def read_raw_data(addr):

    high = bus.read_byte_data(MPU_ADDR, addr)

    low = bus.read_byte_data(MPU_ADDR, addr+1)

    value = (high << 8) | low

    if value > 32768:

        value -= 65536
```

return value

Convert to Acceleration (g)

ACCEL_XOUT = 0x3B

ACCEL_YOUT = 0x3D

ACCEL_ZOUT = 0x3F

Ax = read_raw_data(ACCEL_XOUT) / 16384.0

Ay = read_raw_data(ACCEL_YOUT) / 16384.0

Az = read_raw_data(ACCEL_ZOUT) / 16384.0

Logging Accelerometer Data

```
with open("accel_log.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Time", "Ax(g)", "Ay(g)", "Az(g)", "Magnitude(g)"])
    while True:
        Ax = read_raw_data(ACCEL_XOUT) / 16384.0
        Ay = read_raw_data(ACCEL_YOUT) / 16384.0
        Az = read_raw_data(ACCEL_ZOUT) / 16384.0
        mag = math.sqrt(Ax**2 + Ay**2 + Az**2)
        writer.writerow([time.time(), Ax, Ay, Az, mag])
        print(Ax, Ay, Az, mag)
        time.sleep(0.1)
```

Connection Schematic:

Exercise No.		Date	

Work Space:

Output:

Exercise No.		Date	

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Real time location coordinates logging using GPS module through Raspberry-Pi5

Objective:

To log real-time GPS coordinates using Raspberry Pi 5.

Hardware and Software requirements:

Raspberry Pi 5

GPS Module (NEO-6M), USB, Antenna

Platform: Raspberry Pi OS (64-bit)

Python Libraries: pyserial, pynema2, time, csv

Pre-class exercises:

Topics

- Working principle of GPS
- Latitude and longitude format
- NMEA protocol
- UART communication

Questions

- What is trilateration in GPS?
- What is an NMEA sentence?
- Difference between GGA and RMC sentences?
- Why UART is preferred for GPS modules?

Exercise No.		Date	

In-Lab Exercise:

a) Connection Description

- GPS TX → Raspberry Pi RX (UART)
- GPS RX → Raspberry Pi TX (optional)
- GPS VCC → 5V / 3.3V
- GPS GND → GND

b) Detailed Algorithm

1. Enable UART on Raspberry Pi
2. Initialize serial communication
3. Continuously read GPS serial data
4. Identify valid NMEA sentences
5. Extract latitude and longitude
6. Display coordinates on terminal
7. Log coordinates into CSV file with timestamp

c) Important Code Snippets

Reading GPS Data

```
import serial
import pynmea2
import time
import csv
port = "/dev/serial0"
ser = serial.Serial(port, baudrate=9600, timeout=1)
```

Parsing NMEA Sentences

```
with open("gps_log.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Time", "Latitude", "Longitude"])
    while True:
        data = ser.readline().decode('ascii', errors='replace')
        if data.startswith('$GPGGA'):
            msg = pynmea2.parse(data)
```

```
    if msg.latitude and msg.longitude:
        writer.writerow([time.time(), msg.latitude, msg.longitude])
        print(msg.latitude, msg.longitude)
```

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Voice command-based cruise speed control using VOSK language model through Raspberry-Pi5

Objective:

To implement offline voice command-based cruise speed control using the VOSK speech recognition engine on Raspberry Pi 5.

Hardware and Software requirements:

Raspberry Pi 5

USB Headset

Platform: Raspberry Pi OS (64-bit)

Python Libraries: vosk, pyaudio,json,re

Pre-class exercises:

Topics

- Speech recognition fundamentals
- Offline vs online ASR
- Command grammar

Questions

- Why offline voice recognition is required in vehicles?
- What is keyword spotting?
- How noise affects speech recognition?

Exercise No.		Date	

In-Lab Exercise:

a) System Description

Voice command → speech recognition → text parsing → cruise speed update → motor controller.

b) Detailed Algorithm

- Initialize microphone and audio stream
- Load VOSK language model
- Continuously listen for voice commands
- Convert speech to text
- Identify valid commands
- Extract speed value
- Update cruise speed variable
- Display speed on terminal

c) Important Code Snippets

Initialize VOSK

```
from vosk import Model, KaldiRecognizer

import pyaudio

import json

import re

model = Model("model")

recognizer = KaldiRecognizer(model, 16000)
```

Audio Capture and Recognition

```
p = pyaudio.PyAudio()

stream = p.open(format=pyaudio.paInt16,

                channels=1,

                rate=16000,

                input=True,

                frames_per_buffer=8000)

stream.start_stream()
```

Command Processing

```
while True:

   data = stream.read(4000, exception_on_overflow=False)

   if recognizer.AcceptWaveform(data):
```

```
result = json.loads(recognizer.Result())
text = result.get("text", "")
print("Command:", text)
match = re.search(r"set speed (\d+)", text)
if match:
    cruise_speed = int(match.group(1))
    print("Cruise Speed Set to:", cruise_speed)
```

Connection Schematic:

Work Space:

Exercise No.		Date	

Output:

Exercise No.		Date	

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured: _____out of 50**
	Signature of the Evaluator with Date

Exercise No.		Date	

Additional Practice Exercises

1. **Raspberry Pi Fundamentals & GPIO**
   Modify the GPIO program to implement a traffic signal controller using three LEDs with configurable timing and add keyboard-based control to switch between automatic and manual modes.
2. **IR Sensor–Based Relay Control**
   Enhance the IR–relay system by introducing software debounce logic and implement a safety interlock mechanism that prevents relay toggling under unstable sensor conditions.
3. **Ultrasonic Distance Measurement with GUI**
   Extend the ultrasonic distance measurement system to log distance values with timestamps and generate a graphical plot showing object distance variation over time.
4. **Ultrasonic-Based DC Motor Direction and Speed Control**
   Implement a gradual acceleration and braking strategy using PWM control and integrate a secondary sensor to demonstrate basic sensor-fusion–based obstacle avoidance.
5. **Image and Video Processing using OpenCV**
   Compare multiple edge detection techniques (Canny, Sobel, Laplacian) on live video input and analyze their performance in terms of noise sensitivity and processing speed.
6. **Object Recognition using YOLO**
   Train a custom YOLO model to detect a limited set of objects relevant to road environments and evaluate detection accuracy under varying lighting conditions.
7. **Traffic Sign Recognition using YOLO**
   Integrate detected traffic signs with vehicle logic by dynamically adjusting vehicle speed or triggering alerts based on recognized sign categories.
8. **Lane Detection using Classical OpenCV Techniques**
   Improve lane detection robustness by implementing region-of-interest optimization and estimating the lane center to assist steering decision-making.
9. **Route Visualization using OpenStreetMap**
   Extend the route visualization program to divide the planned path into fixed-distance waypoints and export these waypoints for use in autonomous navigation logic.
10. **GPS-Based Real-Time Location Logging**
    Combine GPS coordinate logging with map visualization to display the vehicle's live position on an OpenStreetMap route and compute the total distance traveled.
11. **Accelerometer Data Acquisition and Logging**
    Analyze accelerometer data to detect sudden vertical acceleration spikes and classify road surface conditions such as smooth road, speed breaker, and pothole.
12. **Pothole Detection using OpenCV**
    Fuse camera-based pothole detection with accelerometer data to reduce false detections and log confirmed pothole locations along with GPS coordinates.
13. **Voice Command Based Cruise Speed Control**
    Enhance the voice control system by adding wake-word detection, multilingual command support, and integration with motor speed control for complete hands-free cruise operation.

Project Problem Statements

Exercise No.		Date	

PROJECT 1: Voice-Enabled Obstacle-Aware Autonomous Rover

Problem Statement

The objective of this project is to design and develop an autonomous rover capable of navigating in indoor or semi-outdoor environments by detecting obstacles in real time and controlling motor speed accordingly, while also allowing the user to override motion commands using offline voice commands. The system should continuously sense obstacles using distance sensors, adjust speed and direction using PWM-controlled motors, and respond immediately to voice commands such as *"stop"*, *"slow down"*, or *"resume"*. The project emphasizes safety, autonomy, and human–machine interaction, which are critical aspects of real-world autonomous vehicle systems.

Tools and Platforms

Hardware: Raspberry Pi 5, Ultrasonic sensor (HC-SR04), DC motors, motor driver (L298N/L293D), USB microphone, chassis, power supply.

Software & Libraries: Python, gpiozero, RPi.GPIO, time, OpenCV (optional for future extension), vosk, pyaudio, re.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The project begins by interfacing ultrasonic sensors with the Raspberry Pi to continuously measure obstacle distance. Based on predefined distance thresholds, the rover's motors are controlled using PWM to regulate speed and direction. In parallel, a voice recognition pipeline is implemented using the VOSK offline speech recognition engine. Voice commands are parsed and mapped to vehicle actions, with safety logic ensuring that voice-based emergency stop commands override all autonomous decisions. The final phase integrates both subsystems into a unified control loop with priority handling and real-time response.

Open-Source Support

- VOSK offline speech recognition models
- Python GPIO libraries
- Community examples for ultrasonic navigation
- OpenCV extensions for future vision integration

Skills Development

Students gain strong skills in embedded Python, real-time motor control, offline speech recognition, sensor fusion fundamentals, and safety-critical system design. This project directly supports roles in robotics, embedded systems, and autonomous platforms

Exercise No.		Date	

PROJECT 2: Vision-Based Lane Following Vehicle with Adaptive Speed Control

Problem Statement

This project aims to develop an autonomous vehicle prototype that detects road lane boundaries using camera input and classical computer vision techniques, and dynamically adjusts vehicle speed based on lane geometry and obstacle proximity. The system must process live video frames, extract lane features, estimate lane curvature, and control motor speed to ensure stable navigation. The emphasis is on perception-driven control without reliance on pre-mapped routes.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, DC motors, motor driver, ultrasonic sensor.

Software & Libraries: Python, OpenCV, NumPy, gpiozero

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The camera feed is captured and preprocessed using grayscale conversion, Gaussian blurring, and edge detection. A region of interest is applied to isolate the road area, followed by Hough transform-based line detection. Lane center estimation is computed, and steering decisions are derived accordingly. Motor speed is adaptively controlled based on lane curvature and obstacle distance, ensuring smooth navigation and safety.

Open-Source Support

- OpenCV lane detection tutorials
- GitHub repositories for classical lane detection
- Python motor control libraries

Skills Development

This project builds strong foundations in computer vision, image processing, control logic, and perception-based navigation—core competencies for autonomous vehicle engineers.

Exercise No.		Date	

PROJECT 3: Traffic Sign Recognition and Speed Enforcement System

Problem Statement

The goal of this project is to design an autonomous system that detects and recognizes traffic signs using deep learning and automatically enforces vehicle behavior such as speed limiting or stopping. The system should identify traffic signs from live camera input and translate recognized signs into real-time motor control decisions, demonstrating intelligent rule-based autonomy.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, motors, motor driver.

Software & Libraries: Python, YOLO (Ultralytics), OpenCV, NumPy.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

A YOLO-based object detection model is deployed on the Raspberry Pi to detect traffic signs. Detected signs are classified and mapped to predefined actions (e.g., speed limit enforcement, stop). The motor control system adjusts PWM signals accordingly. Performance is evaluated under varying lighting and camera angles.

Open-Source Support

- Ultralytics YOLO models
- Traffic sign datasets (GTSRB, Indian sign datasets)
- OpenCV visualization tools

Skills Development

Students develop expertise in deep learning inference, perception-to-control pipelines, and embedded AI deployment—highly valued in AV and AI roles.

Exercise No.		Date	

PROJECT 4: Smart Road Condition Monitoring using Vision and Accelerometer Fusion

Problem Statement

This project focuses on detecting road surface anomalies such as potholes by combining camera-based image processing with accelerometer vibration analysis. The objective is to reduce false detections and improve reliability by validating visual pothole detections using inertial sensor data.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, MPU6050 accelerometer, GPS module (optional).

Software & Libraries: Python, OpenCV, NumPy, smbus2.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

Camera images are processed to identify pothole-like regions using thresholding and contour analysis. Simultaneously, accelerometer data is monitored for vertical acceleration spikes. A fusion logic confirms potholes only when both vision and inertial cues align. Confirmed events are logged for analysis.

Open-Source Support

- OpenCV contour analysis examples
- MPU6050 Python drivers
- Sensor fusion references

Skills Development

This project strengthens understanding of sensor fusion, perception reliability, signal processing, and real-world data validation techniques.

Exercise No.		Date	

PROJECT 5: GPS-Guided Autonomous Navigation with Route Visualization

Problem Statement

The objective of this project is to create a navigation system that integrates OpenStreetMap data with GPS logging to visualize planned routes and track vehicle movement in real time. The system should generate an interactive map showing start and destination points, plot the route, and log intermediate coordinates for autonomous navigation analysis.

Tools and Platforms

Hardware: Raspberry Pi 5, GPS module.

Software & Libraries: Python, OSMnx, Folium, Geopy, pyserial.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

User-defined locations are converted into latitude and longitude coordinates. OpenStreetMap data is used to generate a route, which is displayed on a web-based interactive map. GPS data is continuously logged and overlaid onto the planned route to evaluate navigation accuracy.

Open-Source Support

- OpenStreetMap
- OSMnx and Folium libraries
- Geopy geocoding service

Skills Development

Students gain experience in map integration, localization, real-world data handling, and web-based visualization—critical for autonomous mobility and IoT careers.

Exercise No.		Date	

PROJECT 6: Voice-Controlled Autonomous Delivery Robot with Obstacle Awareness

Problem Statement

The aim of this project is to develop a small autonomous delivery robot capable of navigating a predefined area while responding to user voice commands and avoiding obstacles in real time. The robot should accept offline voice commands such as “start delivery,” “stop,” and “return,” while autonomously controlling motor speed and direction based on obstacle proximity. The system emphasizes safety, autonomy, and natural human–machine interaction, all of which are critical for service robots and last-mile delivery solutions.

Tools and Platforms

Hardware: Raspberry Pi 5, ultrasonic sensor (HC-SR04), DC motors, motor driver, USB microphone, robot chassis, power supply.

Software & Libraries: Python, gpiozero, time, vosk, pyaudio, re.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The project begins by configuring the Raspberry Pi GPIO and motor driver for forward, reverse, and speed control using PWM. Ultrasonic sensors are interfaced to continuously measure distance ahead of the robot. A distance-based control algorithm adjusts motor speed to maintain a safe gap from obstacles, stopping the robot when a critical threshold is reached.

Parallel to this, an offline voice recognition pipeline is implemented using the VOSK language model. The system continuously listens for predefined commands, parses recognized speech, and maps commands to robot actions. A priority-based control logic ensures that voice commands—especially emergency stop—override autonomous navigation decisions.

In the final phase, both subsystems are integrated into a single real-time loop. Extensive testing is performed to validate obstacle avoidance accuracy, voice command responsiveness, and safe system behavior under noisy conditions.

Open-Source Support

- VOSK offline speech recognition models
- Python motor control examples
- Community robotics projects using Raspberry Pi

Skills Development

Students develop skills in embedded programming, real-time control, voice-based interfaces, safety logic design, and system integration—highly relevant for robotics and autonomous systems roles.

Exercise No.		Date	

PROJECT 7: Multi-Sensor Obstacle Avoidance System using Vision and Ultrasonic Sensors

Problem Statement

This project focuses on developing a robust obstacle avoidance system that combines ultrasonic distance sensing with camera-based object detection to improve reliability and decision accuracy. The system should detect obstacles using both sensors and apply fusion logic to control vehicle motion, reducing false detections and improving safety.

Tools and Platforms

Hardware: Raspberry Pi 5, ultrasonic sensor, camera module, DC motors, motor driver.

Software & Libraries: Python, OpenCV, YOLO (Ultralytics), gpiozero.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

Ultrasonic sensors are first configured to measure real-time distance and detect immediate obstacles. In parallel, the camera feed is processed using a YOLO-based object detection model to identify obstacles such as people or vehicles. Sensor data from both sources is combined using a rule-based fusion approach: ultrasonic data ensures immediate collision avoidance, while vision data provides context and classification.

Motor control logic uses the fused decision to adjust speed and direction. The system is tested under different lighting and obstacle configurations to evaluate detection robustness.

Open-Source Support

- Ultralytics YOLO repositories
- OpenCV object detection examples
- Sensor fusion tutorials

Skills Development

This project builds competencies in sensor fusion, embedded AI inference, perception-driven control, and real-time decision-making.

Exercise No.		Date	

PROJECT 8: Autonomous Road Condition Assessment Vehicle using Vision, IMU, and GPS

Problem Statement

The goal of this project is to design an autonomous system that monitors road surface conditions by combining vision-based pothole detection, accelerometer vibration analysis, and GPS-based location logging. The system should detect road anomalies, validate them using inertial data, and record their geographical locations for later analysis.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, MPU6050 accelerometer, GPS module.

Software & Libraries: Python, OpenCV, smbus2, pyserial, pynmea2.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The camera captures road images that are processed to identify pothole-like regions using thresholding and contour analysis. Simultaneously, accelerometer data is monitored for sudden vertical acceleration spikes. A fusion algorithm confirms potholes only when both visual and inertial cues are present. Once confirmed, GPS coordinates are captured and logged along with timestamp and severity.

The collected data is stored in structured files for offline analysis and visualization.

Open-Source Support

- OpenCV contour analysis tutorials
- MPU6050 open-source drivers
- GPS parsing libraries

Skills Development

Students gain hands-on experience in sensor fusion, embedded perception, localization, and data analytics.

Exercise No.		Date	

PROJECT 9: Intelligent Cruise Control System with Obstacle and Voice Awareness

Problem Statement

This project aims to implement an intelligent cruise control system that maintains a user-defined speed while dynamically adjusting vehicle motion based on obstacle proximity and voice commands. The system should safely slow down or stop when obstacles are detected and accept voice commands for speed modification and emergency stop.

Tools and Platforms

Hardware: Raspberry Pi 5, ultrasonic sensor, DC motors, motor driver, USB microphone.

Software & Libraries: Python, gpiozero, vosk, pyaudio.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

Motor control is implemented using PWM to maintain a target cruise speed. Ultrasonic sensors continuously monitor distance ahead, and the control algorithm reduces speed proportionally as obstacles approach. Voice recognition is integrated to allow users to set, increase, or decrease cruise speed and issue emergency stop commands. A hierarchical control structure ensures safety-critical decisions override cruise logic.

Open-Source Support

- VOSK offline ASR
- PWM motor control examples
- Robotics cruise control references

Skills Development

Students develop strong understanding of control logic, safety-critical systems, and human-machine interaction.

Exercise No.		Date	

PROJECT 10: Autonomous Vehicle Event Data Recorder (Black Box System)

Problem Statement

The objective of this project is to develop an event data recorder that continuously logs vehicle state information such as GPS location, acceleration, speed, and detected events. The system acts as a black box for autonomous vehicles, providing valuable data for analysis, debugging, and safety evaluation.

Tools and Platforms

Hardware: Raspberry Pi 5, GPS module, MPU6050 accelerometer, ultrasonic sensor.

Software & Libraries: Python, pynmea2, smbus2, csv.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The system continuously reads sensor data from GPS, accelerometer, and ultrasonic sensors. Events such as sudden braking, collisions, or pothole encounters are detected based on predefined thresholds. All data is time-stamped and logged into structured files for post-event analysis. Optional visualization can be added for better interpretation.

Open-Source Support

- Open-source GPS and IMU libraries
- Vehicle data logging frameworks

Skills Development

This project strengthens skills in data logging, system monitoring, sensor integration, and real-world system validation.

Exercise No.		Date	

PROJECT 11: Real-Time Driver Assistance System (ADAS) Prototype using Vision and Sensors

Problem Statement

This project aims to design and implement a Real-Time Driver Assistance System (ADAS) prototype that assists a human driver by continuously monitoring the driving environment and providing timely alerts. The system should detect lane boundaries, recognize traffic signs, and identify obstacles in real time using camera input and distance sensors. Instead of fully controlling the vehicle, the system focuses on *assistance* by generating visual and audible alerts whenever unsafe driving conditions are detected, such as lane departure, approaching obstacles, or speed limit violations. This project introduces students to the foundational concepts behind modern ADAS technologies used in commercial vehicles.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, ultrasonic sensor, display/monitor, speaker or buzzer.

Software & Libraries: Python, OpenCV, YOLO (Ultralytics), NumPy, gpiozero.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The project begins with capturing real-time video from the camera and processing frames using classical computer vision techniques to detect lane markings. Lane deviation is estimated by calculating the offset between the vehicle's center and lane center. In parallel, a YOLO-based deep learning model is deployed to detect traffic signs such as stop signs or speed limit signs. Ultrasonic sensors continuously monitor the distance to obstacles ahead.

All perception outputs are combined in a decision module that evaluates driving safety conditions. When a risky condition is detected—such as lane departure, obstacle proximity, or speed violation—the system generates visual alerts on the display and audio alerts through a buzzer or speaker. The system is tested under various scenarios to validate responsiveness and reliability.

Open-Source Support

- OpenCV lane detection examples
- Ultralytics YOLO models
- ADAS research demos and datasets

Skills Development

Students develop skills in real-time perception, multi-sensor integration, alert generation, and system-level thinking. This project closely aligns with roles in ADAS development and automotive perception systems.

Exercise No.		Date	

PROJECT 12: Autonomous Campus Shuttle with Map-Based Navigation and Obstacle Handling

Problem Statement

The objective of this project is to develop a scaled autonomous campus shuttle prototype that navigates between predefined locations using digital map data while handling real-time obstacles and logging its movement. The system should visualize routes using OpenStreetMap, follow waypoints, avoid obstacles using sensors, and record its GPS-based trajectory. This project introduces students to map-assisted autonomy and navigation workflows used in real autonomous vehicles.

Tools and Platforms

Hardware: Raspberry Pi 5, GPS module, ultrasonic sensor, DC motors, motor driver, camera (optional).

Software & Libraries: Python, OSMnx, Folium, Geopy, gpiozero, pyserial.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The project starts by allowing the user to define shuttle stops using addresses or coordinates. OpenStreetMap data is used to generate a route and intermediate waypoints. The planned route is visualized on a web-based map. The shuttle then attempts to follow the waypoints using GPS feedback, continuously updating its position.

Ultrasonic sensors monitor the environment to detect obstacles along the route. When obstacles are detected, the shuttle temporarily alters its motion (slows down or stops) and resumes navigation once the path is clear. GPS data is continuously logged to evaluate navigation accuracy and route adherence.

Open-Source Support

- OpenStreetMap and OSMnx
- GPS parsing libraries
- Autonomous navigation tutorials

Skills Development

Students gain practical experience in localization, map integration, waypoint navigation, and safety handling—key competencies for autonomous mobility engineers.

Exercise No.		Date	

PROJECT 13: Vision-Only Autonomous Navigation System

Problem Statement

This project challenges students to design an autonomous navigation system that relies primarily on camera input without the use of distance sensors. The system must detect lanes, identify obstacles, and make navigation decisions solely based on visual information. The goal is to push students toward perception-driven autonomy and help them understand the limitations and strengths of vision-only systems.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, DC motors, motor driver.

Software & Libraries: Python, OpenCV, YOLO (Ultralytics), NumPy.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The camera feed is processed in real time to detect lane markings and estimate vehicle position within the lane. Object detection is used to identify obstacles and classify them. A vision-based decision logic estimates safe navigation paths and controls motor speed and direction accordingly. Extensive testing is conducted to analyze system behavior under varying lighting conditions and cluttered environments.

Open-Source Support

- OpenCV vision pipelines
- YOLO object detection models
- Autonomous navigation research projects

Skills Development

This project builds strong skills in computer vision, deep learning inference, perception-driven control, and system robustness analysis.

Exercise No.		Date	

PROJECT 14: Intelligent Traffic Rule Violation Detection and Logging System

Problem Statement

The objective of this project is to design a system that automatically detects traffic rule violations such as overspeeding and failure to stop at stop signs. The system should analyze visual inputs and vehicle motion data to identify violations and log them for later analysis. This project introduces students to automated traffic monitoring and enforcement concepts.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, GPS module, DC motors.

Software & Libraries: Python, OpenCV, YOLO, pyserial.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The system uses camera input to detect traffic signs such as stop signs and speed limits. Motor speed data is monitored and compared against detected speed limits. GPS data is used to log violation locations. When a violation is detected, the event is logged with timestamp, location, and evidence frame.

Open-Source Support

- Traffic sign datasets
- YOLO detection repositories
- GPS visualization tools

Skills Development

Students gain experience in rule-based reasoning, data logging, computer vision, and smart city technologies.

Exercise No.		Date	

PROJECT 15: Hybrid Simulation–Hardware Autonomous Vehicle Development Platform

Problem Statement

This project aims to develop a hybrid autonomous vehicle platform where navigation and decision-making logic are first tested in simulation and then deployed on a real Raspberry Pi–based vehicle. The system demonstrates how simulation can accelerate development and reduce hardware risks, mirroring industry practices.

Tools and Platforms

Hardware: Raspberry Pi 5, DC motors, GPS module.

Software & Libraries: Python, OSMnx, Folium, OpenCV (optional)..

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

Students first implement navigation logic using map data and simulated inputs. Once validated, the same logic is deployed onto hardware. GPS feedback is used to compare planned vs actual motion. Iterative refinement improves accuracy and reliability.

Open-Source Support

- OpenStreetMap
- Robotics simulation examples
- Python navigation frameworks

Skills Development

This project develops strong skills in system modeling, simulation, real-world deployment, and iterative engineering workflows.

Exercise No.		Date	

PROJECT 16: Road Safety Analytics Platform using Autonomous Mobile Sensing Unit

Problem Statement

The objective of this project is to design an autonomous mobile sensing unit that continuously evaluates road safety conditions by collecting and correlating visual, inertial, and positional data. The system should autonomously move along a road segment, detect potholes and road irregularities, record vibration levels using an accelerometer, and tag all events with GPS coordinates. The collected data should be structured in a way that enables road safety analytics such as identifying high-risk road segments and severity levels. This project focuses on data-driven infrastructure intelligence, a growing requirement in smart cities and autonomous mobility ecosystems.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, MPU6050 accelerometer, GPS module, ultrasonic sensor, DC motors, motor driver.

Software & Libraries: Python, OpenCV, smbus2, pyserial, pynmea2, csv, numpy.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The mobile unit is configured to move at a constant low speed using PWM-controlled motors. Ultrasonic sensors ensure collision-free motion. The camera continuously captures road images, which are processed to detect pothole-like regions using contour and texture analysis. Simultaneously, accelerometer data is monitored to detect sudden vertical acceleration spikes indicating road surface anomalies. A validation logic confirms road defects only when both visual and inertial cues are present.

Once confirmed, GPS data is read and used to tag the event with precise latitude and longitude. All detected events are logged with timestamp, severity, and location into structured files. Post-processing scripts can analyze the data to identify frequently damaged road segments.

Open-Source Support

- OpenCV-based road anomaly detection projects
- MPU6050 Python drivers
- OpenStreetMap for visualization

Skills Development

Students develop skills in sensor fusion, embedded data analytics, autonomous data collection, and smart infrastructure monitoring—highly relevant to IoT and smart mobility roles.

Exercise No.		Date	

PROJECT 17: Autonomous Vehicle with Safety-Critical Emergency Voice Override System

Problem Statement

This project aims to design an autonomous vehicle control system that prioritizes safety by incorporating a safety-critical emergency voice override mechanism. The vehicle should operate autonomously under normal conditions but must immediately halt all motion when a predefined emergency voice command is detected, regardless of sensor or navigation state. This project introduces students to functional safety concepts used in automotive systems.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, MPU6050 accelerometer, GPS module, ultrasonic sensor, DC motors, motor driver.

Software & Libraries: Python, vosk, pyaudio, gpiozero, re.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The autonomous control logic handles normal motion based on obstacle detection. In parallel, an always-listening voice recognition pipeline runs in a high-priority thread. The system uses a predefined grammar to detect emergency keywords such as “stop immediately” or “emergency brake.” Upon detection, the emergency handler interrupts all other processes and forces motor shutdown.

Extensive testing is conducted to ensure that emergency commands are recognized reliably even during motion and environmental noise. The system logs emergency events for later analysis.

Open-Source Support

- VOSK offline ASR models
- Robotics safety case studies
- Python concurrency examples

Skills Development

Students gain exposure to safety-critical design, priority handling, concurrency, and fault-tolerant control—key skills in automotive and industrial robotics engineering.

Exercise No.		Date	

PROJECT 18: Adaptive Headway and Collision Avoidance Control System

Problem Statement

The objective of this project is to design an adaptive headway control system that maintains a safe distance from leading obstacles by dynamically controlling vehicle speed. The system should continuously estimate distance, adjust speed proportionally, and respond smoothly to changes in obstacle position. This project introduces students to practical control strategies used in adaptive cruise control systems.

Tools and Platforms

Hardware: Raspberry Pi 5, ultrasonic sensor, DC motors, motor driver, MPU6050 accelerometer.

Software & Libraries: Python, gpiozero, smbus2, time.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

The ultrasonic sensor continuously measures the distance to the object ahead. A control algorithm maps distance values to speed setpoints, ensuring gradual deceleration as obstacles approach. Accelerometer data is used to verify smooth acceleration and braking behavior. The system is tuned to avoid jerks and oscillations, mimicking real adaptive cruise control behavior.

Testing involves varying obstacle distances and observing system stability and responsiveness.

Open-Source Support

- Adaptive cruise control tutorials
- Python motor control examples

Skills Development

Students gain hands-on experience in feedback control, system tuning, motion smoothness analysis, and real-time embedded control.

Exercise No.		Date	

PROJECT 19: Autonomous Road Inspection Robot for Industrial and Campus Environments

Problem Statement

This project focuses on developing an autonomous inspection robot that systematically surveys roads in industrial plants or campuses to identify surface defects and obstacles. The robot should autonomously navigate, detect road issues, and generate inspection logs with visual evidence and location data.

Tools and Platforms

Hardware: Raspberry Pi 5, camera module, GPS module, ultrasonic sensor, DC motors.

Software & Libraries: Python, OpenCV, OSMnx, Folium, pyserial.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

Predefined inspection routes are created using map data. The robot follows the route while capturing camera images and checking for potholes. GPS coordinates are logged for each detected issue. Ultrasonic sensors prevent collisions. The final output is a structured inspection report.

Open-Source Support

- OpenStreetMap
- OpenCV inspection demos

Skills Development

Students gain experience in autonomous inspection, mapping, perception, and reporting systems.

Exercise No.		Date	

PROJECT 20: Mini Autonomous Vehicle Stack

Problem Statement

This project aims to design and implement a complete mini autonomous vehicle stack integrating perception, localization, planning, control, and human interaction. The vehicle should navigate mapped routes, detect lanes and obstacles, recognize traffic signs, adapt speed, respond to voice commands, and log sensor data. This project represents a scaled-down but realistic autonomous vehicle system.

Tools and Platforms

Hardware: Raspberry Pi 5, camera, GPS, accelerometer, ultrasonic sensor, microphone, motors.

Software & Libraries: Python, OpenCV, YOLO, OSMnx, Folium, VOSK, gpiozero.

Platform: Raspberry Pi OS (64-bit).

Detailed Work Plan

Subsystems are developed individually and then integrated incrementally. Perception modules detect lanes, obstacles, and signs. Localization combines GPS and map data. Planning logic follows waypoints while avoiding obstacles. Control modules manage motor speed and direction. Voice commands provide human override. Logging ensures traceability and debugging.

Open-Source Support

- Autonomous driving open-source projects
- ROS concepts (optional extension)
- Community AV repositories

Skills Development

This project develops system integration, autonomy stack understanding, embedded AI, and engineering confidence—a powerful differentiator for jobs, internships, and higher studies.

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT DIARY

Exercise No.		Date	

PROJECT REVIEW-1

Progress:

Presentation:

Comments:

Review Marks & Signature:

Exercise No.		Date	

PROJECT REVIEW-2

Progress:

Presentation:

Comments:

Review Marks & Signature: